

WineQuality.Rmd

inteRstellar

13/5/2022

Point 1

Wine Dataset

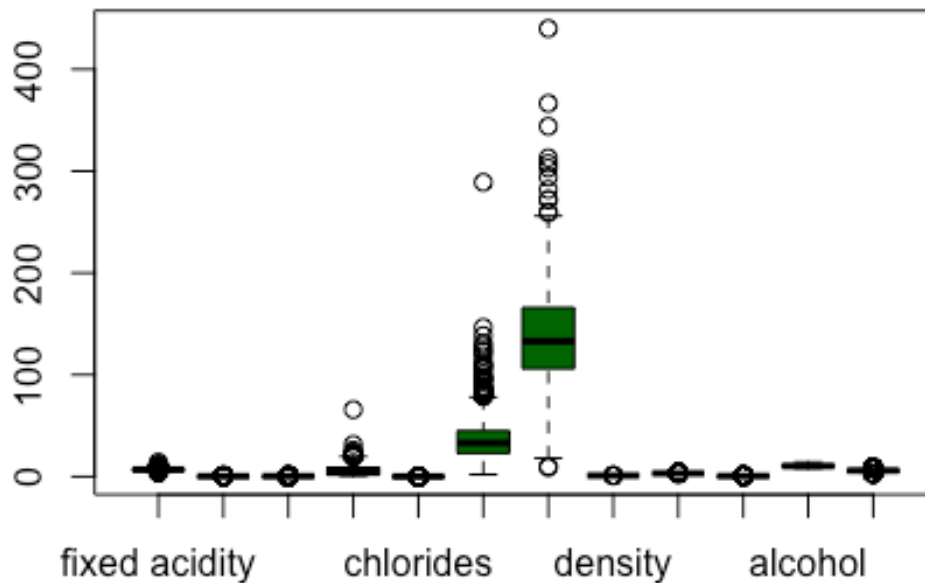
Point 2

Data load

Cleaning the dataset:

-NAs -duplicate -outliers

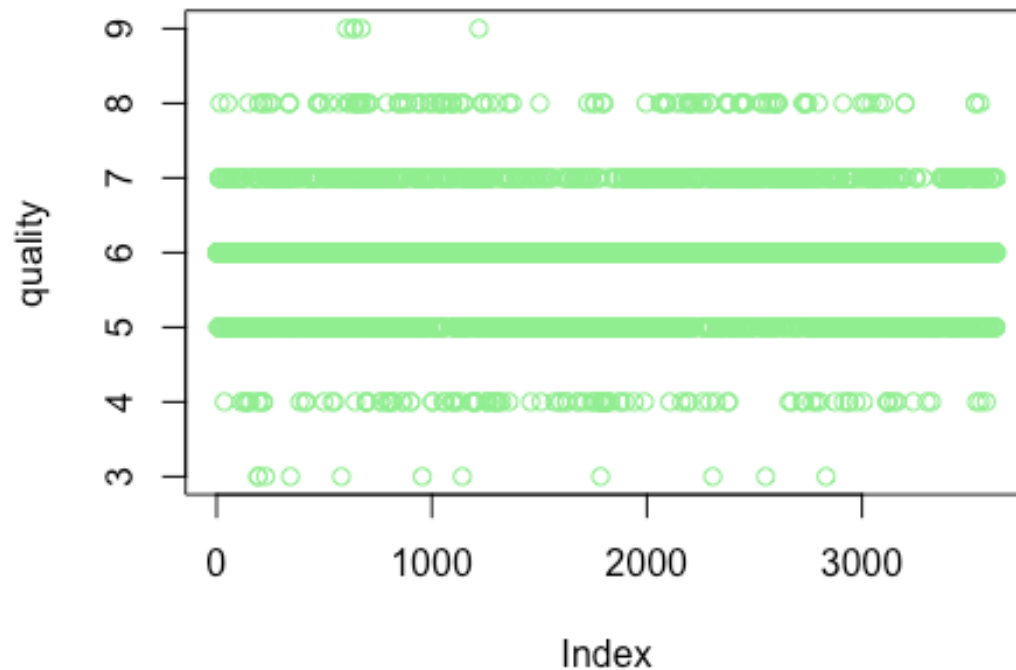
```
#all the variables are numeric  
#omitting the NAs  
data <- na.omit(data)  
#deleting the duplicates  
any(duplicated(data)) #we see that there are some duplicates  
data <- distinct(data) #so we get rid of it  
#Check if there are outliers (yes)  
dim(data) #dim of rows and cols  
boxplot(data, col="darkgreen")
```



```
data <- data.frame(data)
#remove the outliers from every predictor except quality
for(i in colnames(data)[1:11]) {
  print(colnames(data))
  quartiles <- quantile(data[,i], probs=c(.15, .95), na.rm = FALSE)
  IQR <- IQR(data[,i])
  Lower <- quartiles[1] - 1.5*IQR
  Upper <- quartiles[2] + 1.5*IQR
  data<- subset(data, data[,i] > Lower & data[,i] < Upper)
}
```

Insights on data

```
#variable quality seems to be discrete
plot(data$quality, ylab="quality", col="lightgreen")
```



```
table(data$quality)

##
##      3      4      5      6      7      8      9
##    11    116 1042 1661  670  121    5

table(data$quality)/length(data$quality)

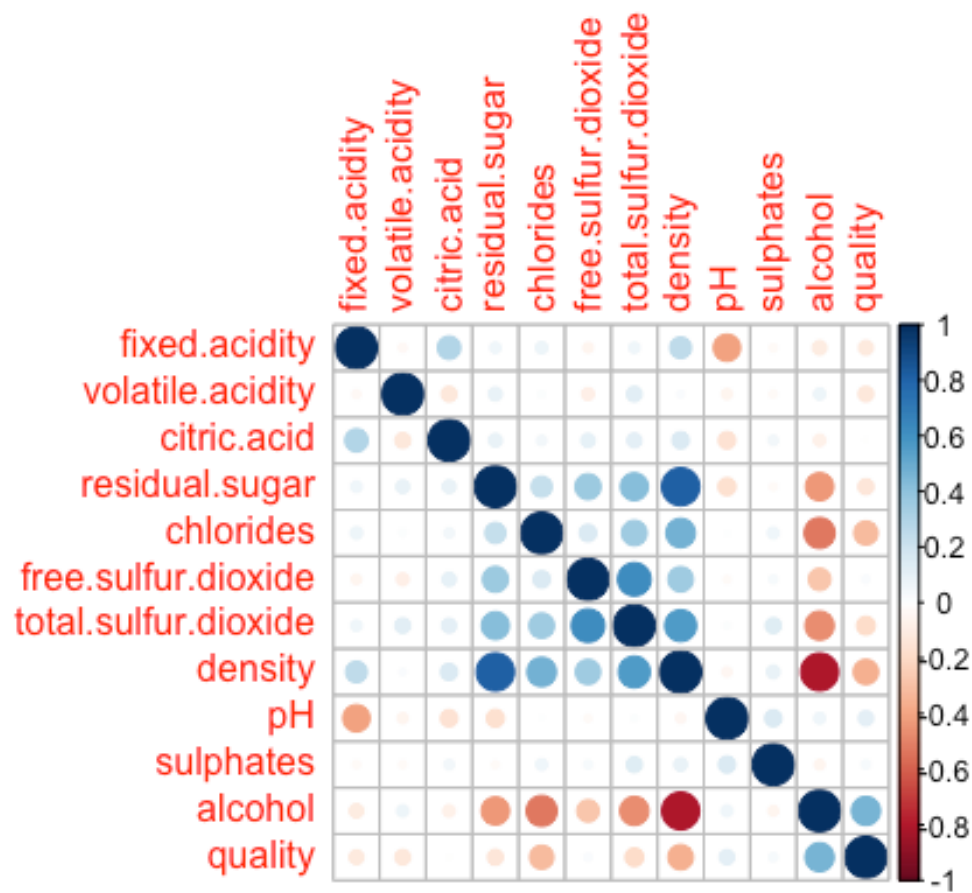
##
##           3           4           5           6           7           8
## 0.003033646 0.031991175 0.287369002 0.458080530 0.184776613 0.033370105
##           9
## 0.001378930
```

We want to look at the most significant variables:

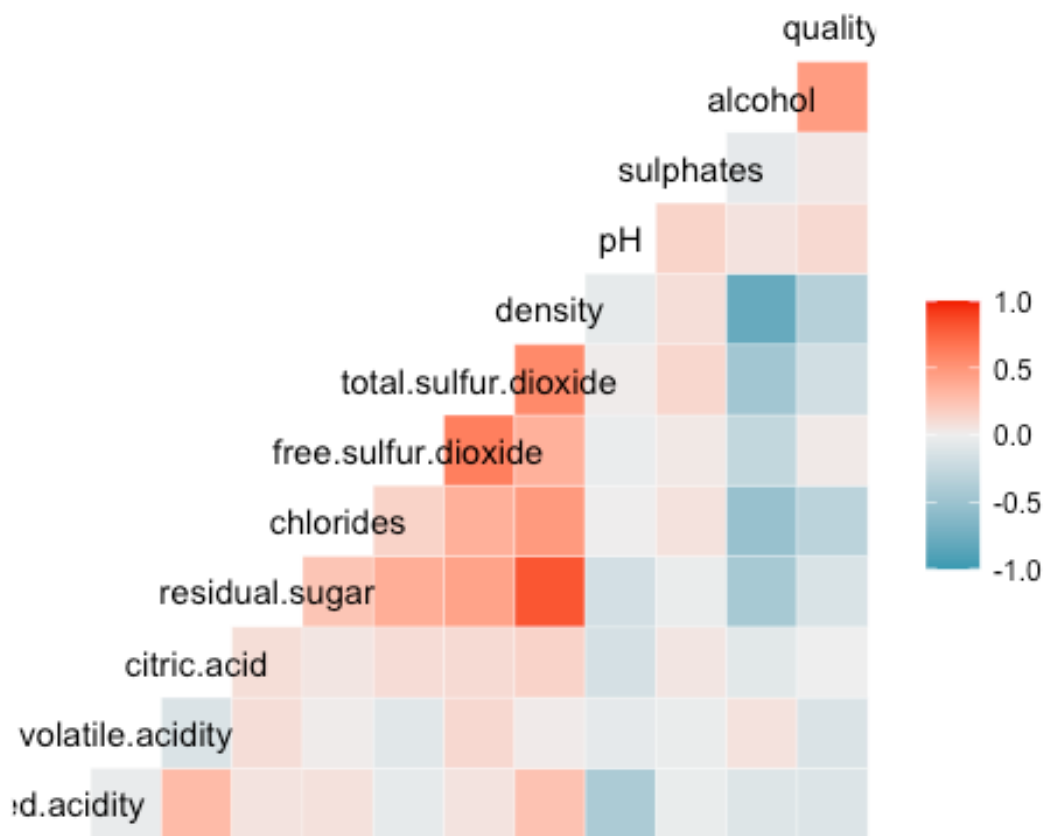
Correlation between variables

We observe that there is very high correlation in absolute values between alcohol and quality, residual sugar and density, alcohol and density

```
data %>% cor() %>% corplot::corrplot()
```



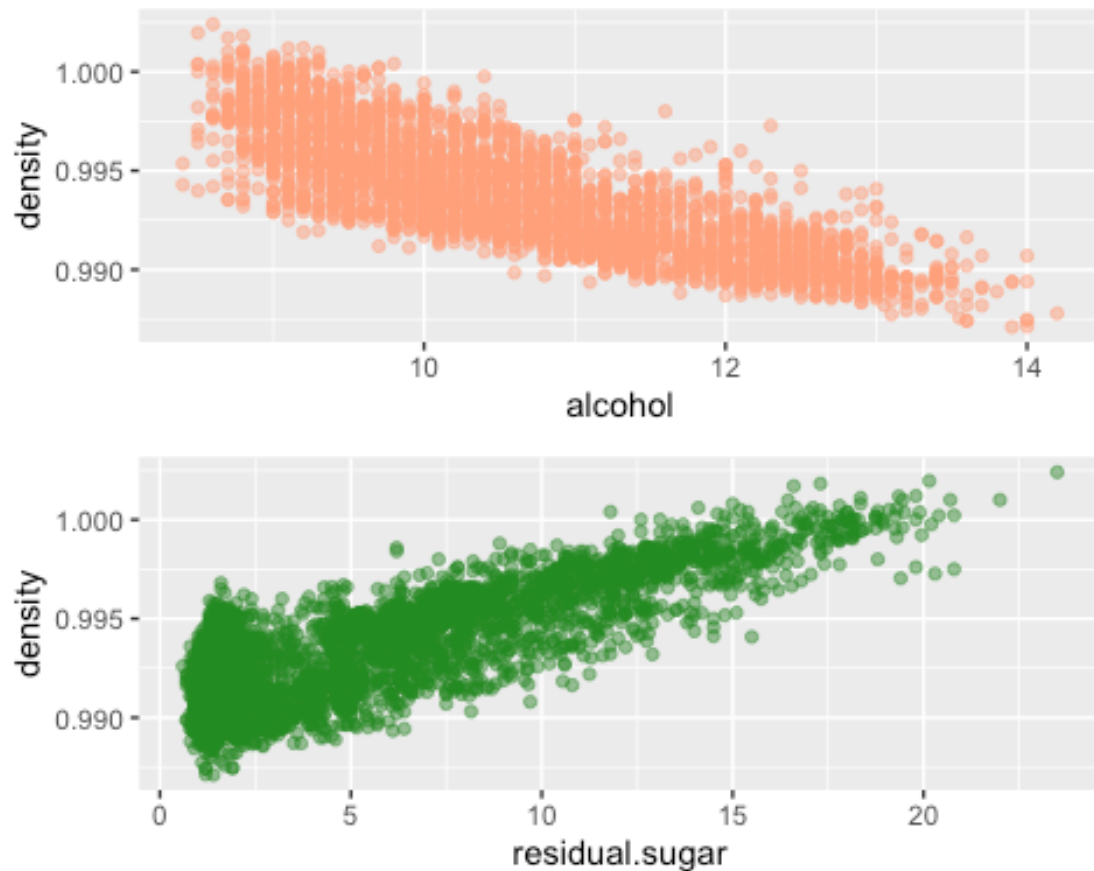
```
data %>% ggcorr()
```



We then plot the relations between alcohol, residual.sugar and density

We observe that density seems to be directly proportional to residual.sugar while alcohol is inversely proportional to density. They also both seem to be linearly dependent to density.

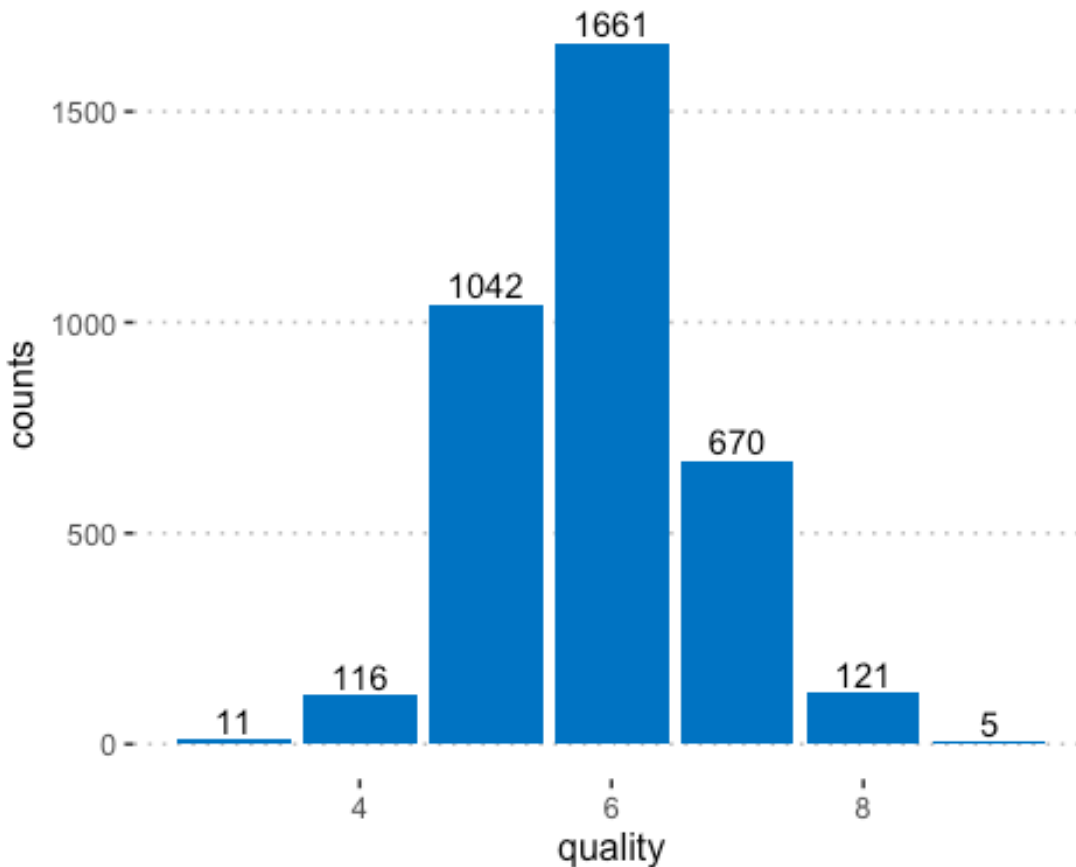
```
p1 = ggplot(data, aes(x=alcohol, y=density))+ geom_point(alpha=0.5,col="light
salmon")
p2 =ggplot(data, aes(x=residual.sugar, y=density))+geom_point(alpha=0.5,col="
forestgreen")
grid.arrange(p1,p2)
```



Barplot to show the distribution of quality in wines:

Most wines have quality 5,6 or 7

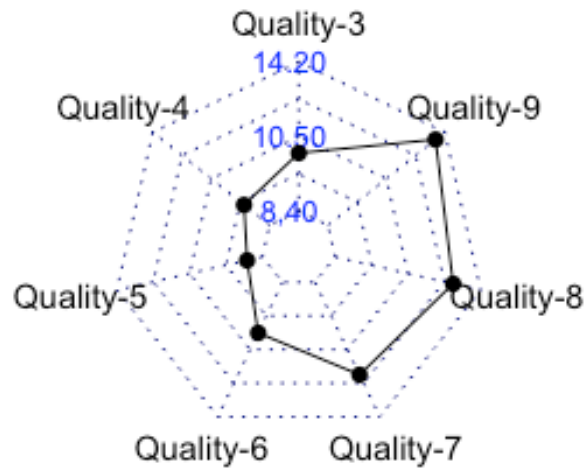
```
data2 <- data.frame(data)
quality_count <- data2 %>%
  group_by(quality) %>%
  summarise(counts = n())
ggplot(quality_count, aes(x = quality, y = counts)) +
  geom_bar(fill = "#0073C2FF", stat = "identity") +
  geom_text(aes(label = counts), vjust = -0.3) +
  theme_pubclean()
```



We confirm that there is a positive linear relationship between alcohol and quality: for high values of quality (6,7,8) there are high values of alcohol

Values of alcohol by quality labels

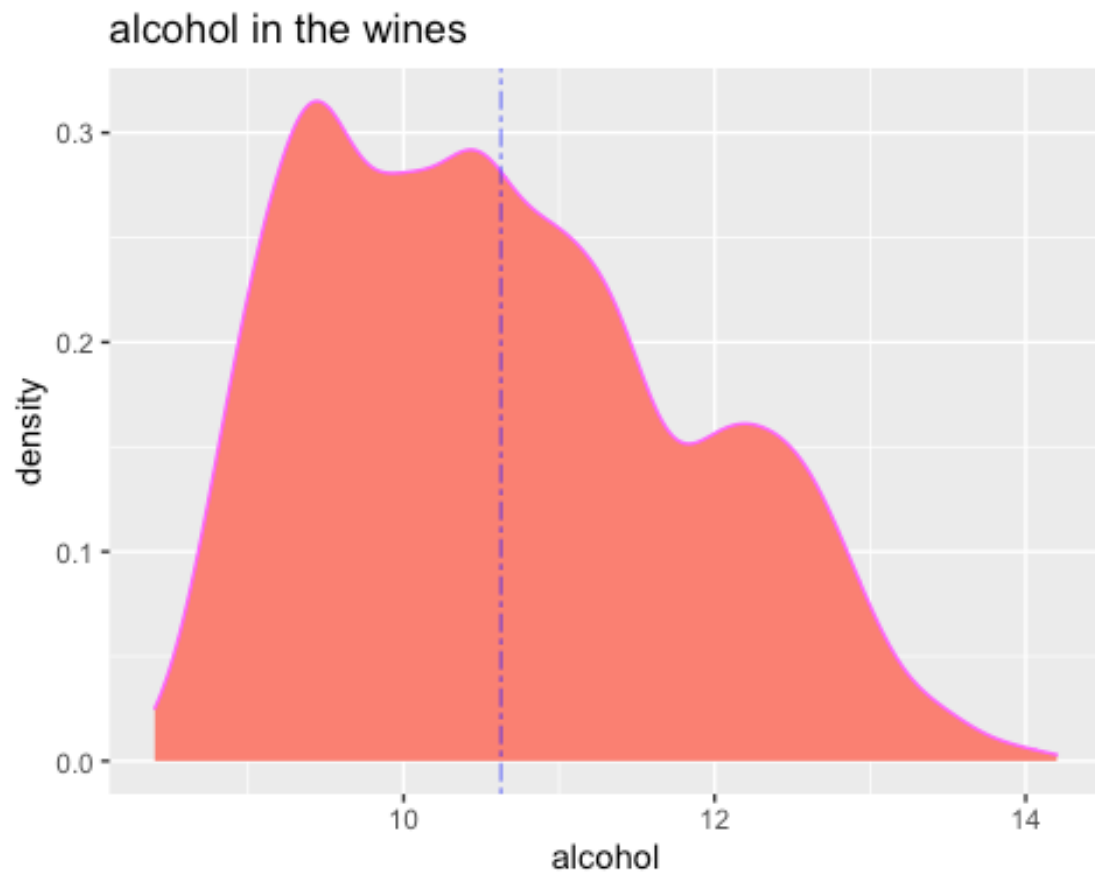
```
options(repr.plot.width=7, repr.plot.height=5.5) #Setting the plot size
alcohol.quality <- data %>%
  group_by(quality) %>%
  summarise(alcohol_mean = mean(alcohol))
alcohol_quality <- as.data.frame(t(alcohol.quality))
names(alcohol_quality) <- paste('Quality',alcohol_quality[1,], sep = '-')
alcohol_quality <- alcohol_quality[2,]
alcohol_quality=rbind(rep(12.4,6) , rep(9.6,6) , alcohol_quality)
radarchart(alcohol_quality, axistype=1, calcex = 0.8, vlcex=0.9,
  caxislabels = c('8,40', '', '10.50', '', '14.20'))
```



Alcohol in wines

We can observe that higher values of alcohol penalize the density of wine.

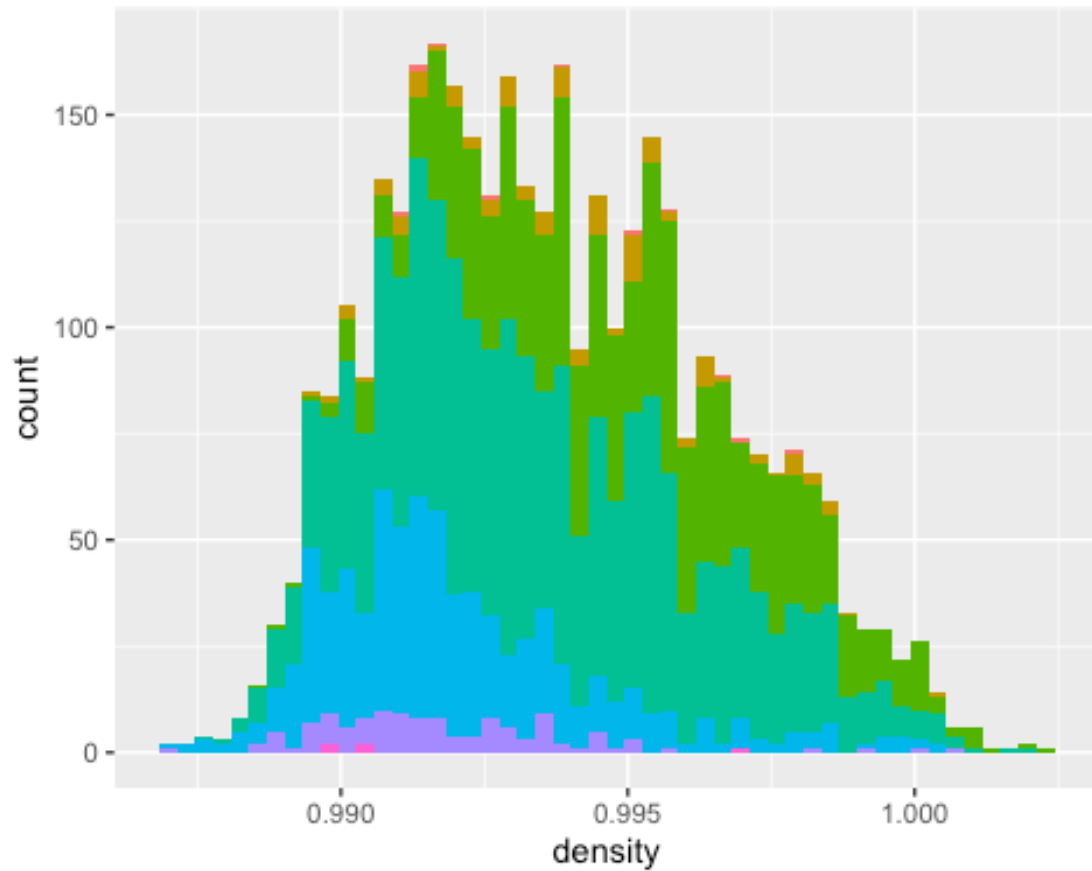
```
curve1 <- ggplot(data, aes(x = alcohol,)) +
  geom_density(color="#FF66FF", fill="salmon") + labs(title="alcohol in the w
ines")
curve1+geom_vline(aes(xintercept=mean(alcohol)),
  color="blue", linetype="twodash", size=0.25)
```

Insights on density:

Most wines tend to have a density value between 0,99 and 1

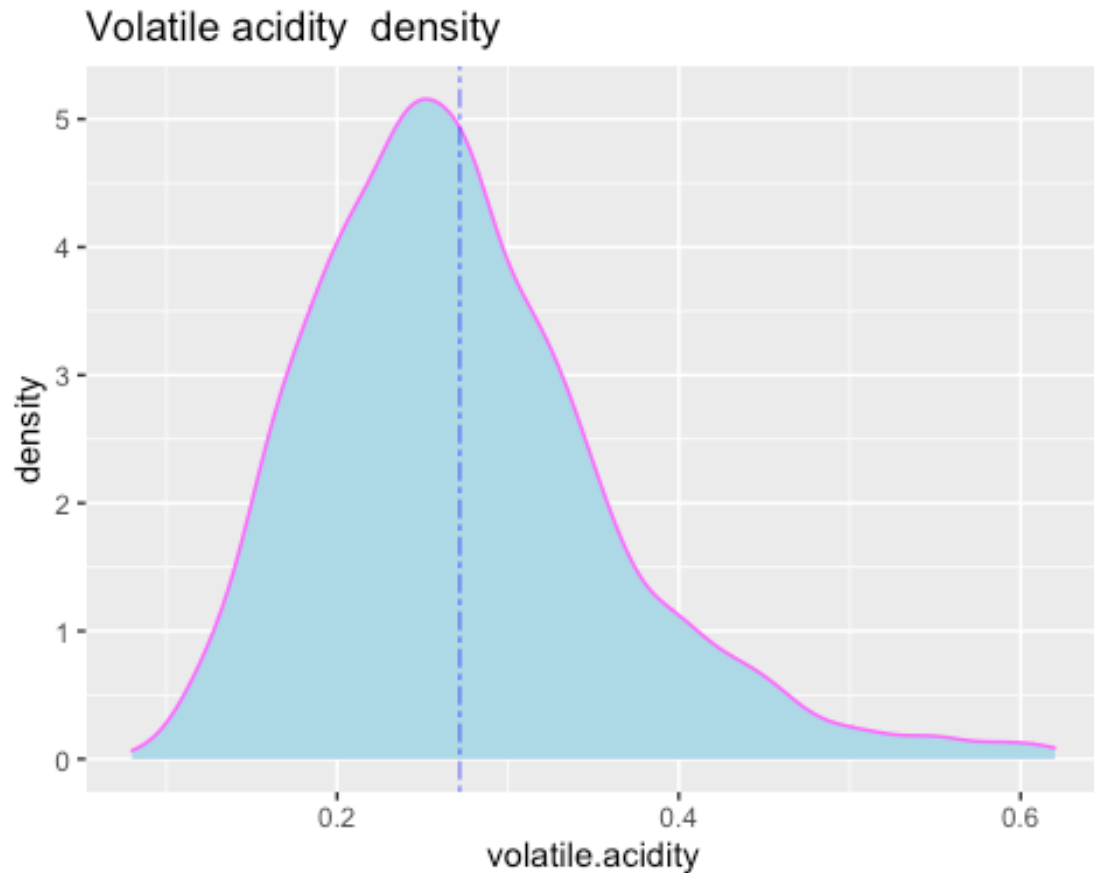
```
data2 %>%  
  ggplot(aes(x = density, fill = cut(quality, 100))) +  
  geom_histogram(show.legend = FALSE, bins=50)
```



Insights on volatile acidity against density:

There seems to be a normal distribution of the volatile acidity against density

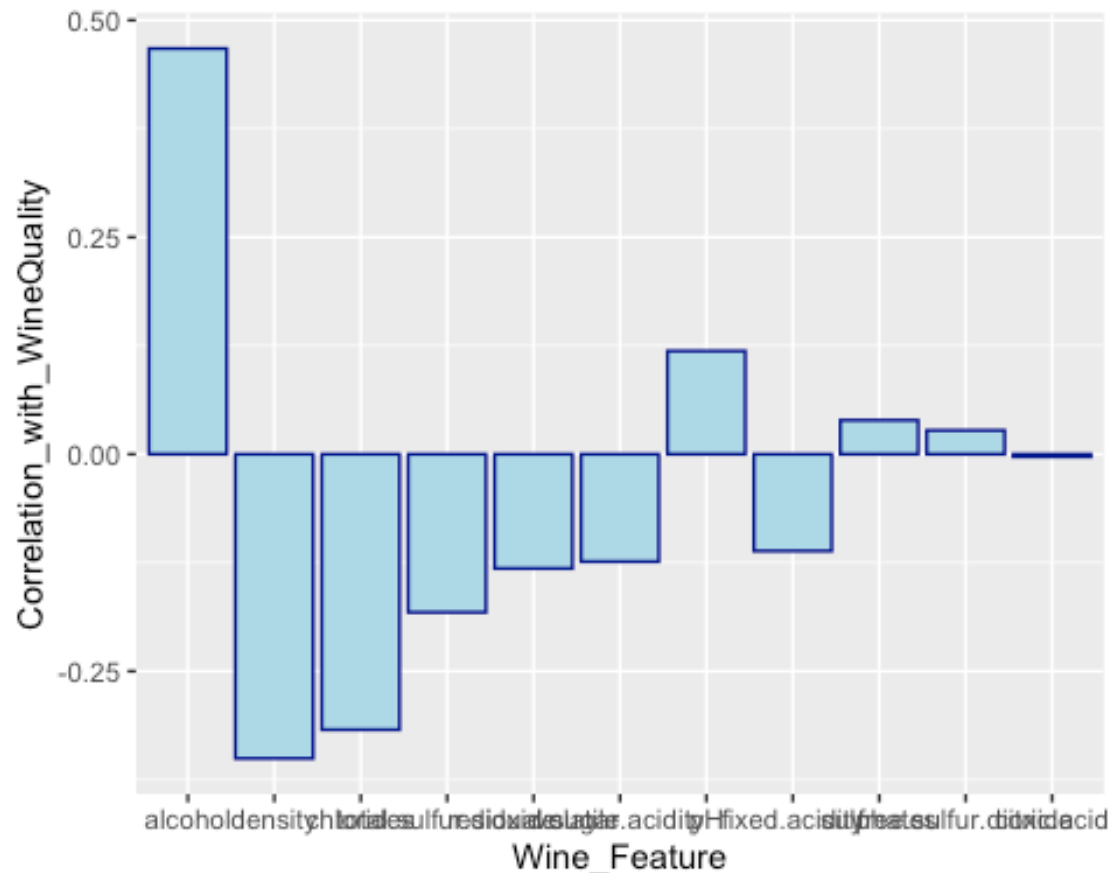
```
curve1<-ggplot(data, aes(x = volatile.acidity)) +geom_density(color="#FF66FF",
, fill="lightblue") + labs(title="Volatile acidity density")
curve1+geom_vline(aes(xintercept=mean(volatile.acidity)),color="blue", linety
pe="twodash", size=0.25)
```



Barplot of correlation between every feature and the wine quality

We can observe that the variables with highest correlation with quality in absolute value are alcohol, density and chlorides

```
##  
## Attaching package: 'data.table'  
## The following objects are masked from 'package:dplyr':  
##  
##   between, first, last  
## The following object is masked from 'package:purrr':  
##  
##   transpose
```



Then we scale the dataset

```
data %<>% scale()
```

Point 3

This dataset contains information about the white variant of the Portuguese Vinho Verde. Wine samples are recorded on the rows. There exist lots of different markets for wines, each differs in quality and consequently in prices. In a business perspective it is undoubtedly important to catch the correct quality of the 'in working wine' and consequently to sell it at the right price. In order to do it we built a regression algorithm that predicts the quality of the wine based on some chemicals predictors. To compare the models we used the MSE since it is very penalizing in case the difference between the right quality and the predicted wine's quality differs a lot.

Point 4

We moreover investigate regarding which of the variables are meaningful by implementing different models and interpreting them.

*the coefficient may vary through different running of the code

Firstly to fit a simple linear model to see at first sight which predictors may be significant

```
linear_model <- lm(quality~., data = data2)
confint(linear_model, level = 0.95)

##                2.5 %          97.5 %
## (Intercept)      1.427886e+02  2.499583e+02
## fixed.acidity      7.421574e-02  1.802663e-01
## volatile.acidity  -1.620138e+00 -1.036979e+00
## citric.acid       -1.841167e-02  5.050329e-01
## residual.sugar     6.750827e-02  1.076132e-01
## chlorides        -7.798881e+00 -2.364676e+00
## free.sulfur.dioxide 5.572170e-03  9.739541e-03
## total.sulfur.dioxide -1.245266e-03  5.287012e-04
## density          -2.525051e+02 -1.438813e+02
## pH                8.893596e-01  1.398506e+00
## sulphates         4.896776e-01  9.657218e-01
## alcohol           6.043171e-02  1.942223e-01
```

From the very beginning we may observe that the not significant predictors are:

total sulfur dioxide and citric acid.

Instead that the predictors with very low p-values and so high significance are: volatile acidity, residual sugar, density and alcohol.

These results can be underlined by the fact that with 95% confidence that there is no evidence of a relationship between citric acid or total.sulfur.dioxide and the wine quality because 0 is included in their confint.

Afterwards we try to fit some models only with the expected significant predictors

Multiple linear regression

```
poly <- lm(quality~alcohol + volatile.acidity, data = data2)
confint(poly, level = 0.95)

##                2.5 %          97.5 %
## (Intercept)      2.4129743  2.8686102
## alcohol           0.3260463  0.3671912
## volatile.acidity -1.8598929 -1.2935705

summary(poly)

##
## Call:
## lm(formula = quality ~ alcohol + volatile.acidity, data = data2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.4185 -0.4824 -0.0232  0.4952  3.1801
##
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.64079    0.11620   22.73  <2e-16 ***
## alcohol           0.34662    0.01049   33.03  <2e-16 ***
## volatile.acidity -1.57673    0.14442  -10.92  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7598 on 3623 degrees of freedom
## Multiple R-squared:  0.2433, Adjusted R-squared:  0.2428
## F-statistic: 582.3 on 2 and 3623 DF, p-value: < 2.2e-16
```

The model is: $E[\text{quality}] = \beta_0 + \beta_1 \text{alcohol} + \beta_2 \text{volatile.acidity} =$

$2,64 + \text{alcohol} * 0,34 - \text{volatile.acidity} * 1,58$ in this case both predictors are significant

We observe that if the alcohol increase of 1 unit the expected quality to increases by around 0,34 , but an increase in volatile.acidity will decrease the expected quality by 1,58
We can be 95% confident that the true value of alcohol is between 0,33 and 0,37 while for volatile.acidity it is between -1,86 and -1,29

Adding an interaction term

```
inter <- lm(quality~ alcohol+ volatile.acidity + volatile.acidity*alcohol, data=data2) #alternative and more compact way
confint(inter, level = 0.95)
```

```
##              2.5 %      97.5 %
## (Intercept)      4.49188074  5.9629644
## alcohol           0.03353786  0.1717454
## volatile.acidity  -12.68980329 -7.9303273
## alcohol:volatile.acidity  0.59940263  1.0441032
```

```
summary(inter)
```

```
##
## Call:
## lm(formula = quality ~ alcohol + volatile.acidity + volatile.acidity *
##      alcohol, data = data2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5379 -0.4771 -0.0047  0.4677  3.1813
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      5.22742    0.37516  13.934  < 2e-16 ***
## alcohol           0.10264    0.03525   2.912  0.00361 **
## volatile.acidity  -10.31007    1.21377  -8.494  < 2e-16 ***
## alcohol:volatile.acidity  0.82175    0.11341   7.246 5.22e-13 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7545 on 3622 degrees of freedom
## Multiple R-squared:  0.2541, Adjusted R-squared:  0.2534
## F-statistic: 411.2 on 3 and 3622 DF,  p-value: < 2.2e-16
```

The model is: $E[\text{quality}] = \text{beta0} + \text{beta1} * \text{alcohol} + \text{beta2} * \text{volatile.acidity} + \text{beta3} * \text{alcohol} * \text{volatile.acidity}$

$5.22 + 0.1 * \text{alcohol} - 10.31 * \text{volatile.acidity} + 0.82 * \text{alcohol} * \text{volatile.acidity}$

Notice that the term B2 is estimated to be negative, so, for example:

An increase of alcohol increases the response of $0.1 + 0.82 * \text{volatile.acidity}$, but when volatile.acidity increases of one unit the response decreases by 10.31 but increases by $0.82 * \text{alcohol}$

Using Gam to individuate the best most meaningful variables

```
ooGam <- gam::gam(quality~., data = data2, control = gam.control(maxit=100))
summary(ooGam)
```

```
##
## Call: gam::gam(formula = quality ~ ., data = data2, control = gam.control(
maxit = 100))
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -3.4298 -0.4835 -0.0194  0.4658  3.0768
##
## (Dispersion Parameter for gaussian family taken to be 0.5362)
##
##      Null Deviance: 2764.177 on 3625 degrees of freedom
## Residual Deviance: 1937.912 on 3614 degrees of freedom
## AIC: 8044.384
##
## Number of Local Scoring Iterations: 2
##
## Anova for Parametric Effects
##
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)	
fixed.acidity	1	34.42	34.417	64.1840	1.515e-15	***
volatile.acidity	1	45.02	45.021	83.9602	< 2.2e-16	***
citric.acid	1	0.70	0.695	1.2966	0.254917	
residual.sugar	1	36.46	36.461	67.9956	2.268e-16	***
chlorides	1	231.71	231.713	432.1209	< 2.2e-16	***
free.sulfur.dioxide	1	18.36	18.365	34.2487	5.282e-09	***
total.sulfur.dioxide	1	36.35	36.346	67.7818	2.523e-16	***
density	1	262.44	262.439	489.4213	< 2.2e-16	***
pH	1	124.52	124.516	232.2098	< 2.2e-16	***
sulphates	1	28.82	28.823	53.7514	2.794e-13	***
alcohol	1	7.47	7.468	13.9263	0.000193	***
Residuals	3614	1937.91	0.536			

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

We can observe that for this model all the predictors are meaningful, except for citric.acid

Point 5

Splitting of the dataset

```
set.seed(123)
idx <- sample(nrow(data), 0.75*nrow(data))
d_train <- data[idx, ]
d_test <- data[-idx,]
d_ft <-data.frame(d_train)
d_ftest <-data.frame(d_test)
test_quality <- d_ftest$quality
summary(d_ftest)
d_ftest1 <- d_ftest
d_ftest = d_ftest %>% dplyr::select(-quality)
```

Subset selection

```
library (leaps)
regfit.full <- regsubsets(quality~., d_ft, nvmax= 11)
summary(regfit.full)

## Subset selection object
## Call: regsubsets.formula(quality ~ ., d_ft, nvmax = 11)
## 11 Variables (and intercept)
##              Forced in Forced out
## fixed.acidity      FALSE      FALSE
## volatile.acidity   FALSE      FALSE
## citric.acid         FALSE      FALSE
## residual.sugar     FALSE      FALSE
## chlorides           FALSE      FALSE
## free.sulfur.dioxide FALSE      FALSE
## total.sulfur.dioxide FALSE      FALSE
## density             FALSE      FALSE
## pH                  FALSE      FALSE
## sulphates           FALSE      FALSE
## alcohol             FALSE      FALSE
## 1 subsets of each size up to 11
## Selection Algorithm: exhaustive
##      fixed.acidity volatile.acidity citric.acid residual.sugar chlori
des
## 1  ( 1 )  " "          " "          " "          " "          " "
## 2  ( 1 )  " "          " "          " "          " "          " "
## 3  ( 1 )  " "          "*"          " "          " "          " "
## 4  ( 1 )  " "          "*"          " "          " "          " "
## 5  ( 1 )  " "          "*"          " "          " "          "*"
## 6  ( 1 )  " "          "*"          " "          "*"          " "
## 7  ( 1 )  "*"          "*"          " "          "*"          " "
```



```
## 8 ( 1 ) "*"          "*"          " "          "*"          "*"
## 9 ( 1 ) "*"          "*"          " "          "*"          "*"
## 10 ( 1 ) "*"         "*"          "*"          "*"          "*"
## 11 ( 1 ) "*"         "*"          "*"          "*"          "*"
##          free.sulfur.dioxide total.sulfur.dioxide density pH sulphates
## 1 ( 1 ) " "          " "          " "          " "          " "
## 2 ( 1 ) "*"          " "          " "          " "          " "
## 3 ( 1 ) "*"          " "          " "          " "          " "
## 4 ( 1 ) "*"          " "          " "          "*"          " "
## 5 ( 1 ) "*"          " "          " "          "*"          " "
## 6 ( 1 ) "*"          " "          "*"          "*"          " "
## 7 ( 1 ) "*"          " "          "*"          "*"          "*"
## 8 ( 1 ) "*"          " "          "*"          "*"          "*"
## 9 ( 1 ) "*"          " "          "*"          "*"          "*"
## 10 ( 1 ) "*"         " "          "*"          "*"          "*"
## 11 ( 1 ) "*"         "*"          "*"          "*"          "*"
##          alcohol
## 1 ( 1 ) "*"
## 2 ( 1 ) "*"
## 3 ( 1 ) "*"
## 4 ( 1 ) "*"
## 5 ( 1 ) "*"
## 6 ( 1 ) "*"
## 7 ( 1 ) " "
## 8 ( 1 ) " "
## 9 ( 1 ) "*"
## 10 ( 1 ) "*"
## 11 ( 1 ) "*"

```

For example we can see that the best 2 variable model contains only volatile.acidity and alcohol.

BIC decreases as the number of variables inside the model are inserted, but it increases again after 7 variables are inserted.

```
par(mfrow = c(2, 2))
plot(reg.summary$bic , xlab = "Number of Variables",
     ylab = " BIC ", type = "l")
which.min(reg.summary$bic)

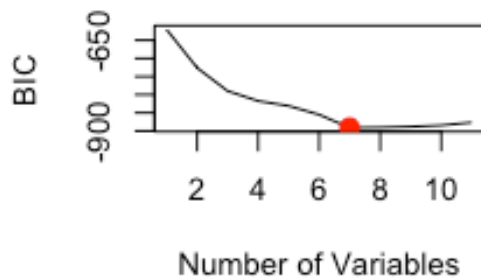
## [1] 7

points(7, reg.summary$bic[7], col = " red ", cex = 2,pch = 20)
coef(regfit.full , 7)

##          (Intercept)          fixed.acidity          volatile.acidity          residual.
sugar
##          -0.008981641          0.189648386          -0.122617418          0.6866
19593
## free.sulfur.dioxide          density          pH          sulp
hates

```

##	0.148904712	-1.009011188	0.260845324	0.1173
71961				



The model with lowest BIC is with 7 variables coefficient.

Stepwise selection through aic and bic

AIC

```
lin = lm(quality~.,data = d_ft)
AIC_both = step(lin,direction = "both", k=2)
```

for AIC metric total.sulfur.dioxide is the only meaningful predictor definitely in contrast with the previous affirmations.

BIC

```
rows = nrow(d_ft)
BIC_both = step(lin,direction = "both", k = log(rows))
linbic = lm(quality~chlorides+citric.acid+total.sulfur.dioxide,data = d_ft)
pred_bic= predict(linbic, d_ftest1)
mse_bic = mean((pred_bic - test_quality)^2)
```

AIC and BIC (fitted with forward and backward selection) suggest to use just the total.sulfur.dioxide to implement the model, BIC uses also citric.acid and chlorides

Cross validation

To choose the dimension for the linear model that minimizes the error, we perform best subset selection on the full data set and select the best variable model. It is important that we make use of the full data set in order to obtain more accurate coefficient estimates.

We perform kfold cross validation to choose among all dimension models and find the one that minimizes the error is the one with 10 variables

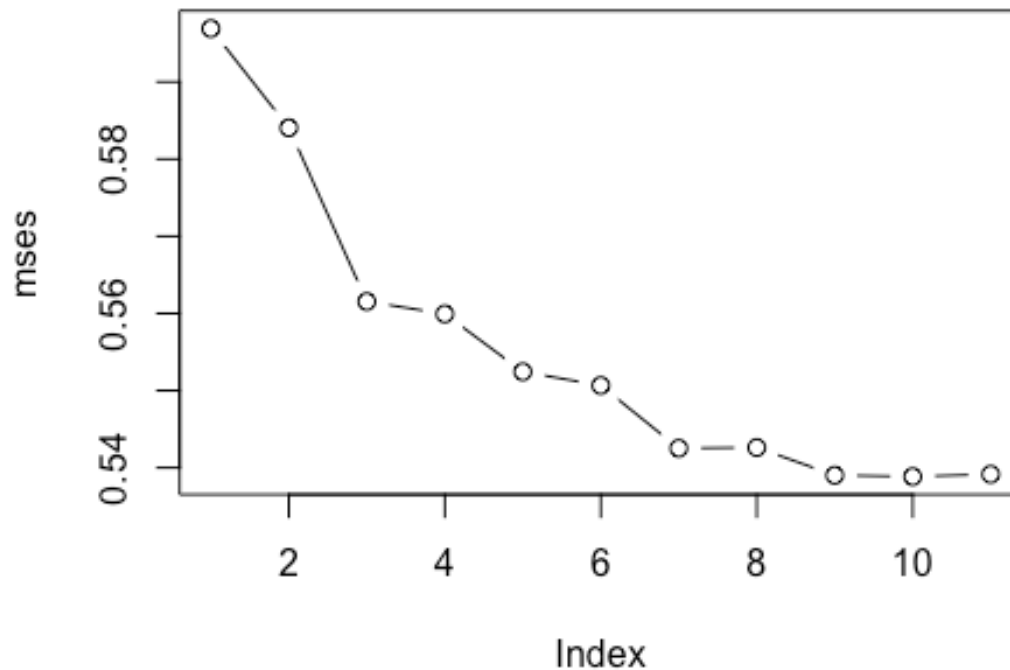
```
k <- 10
n <- nrow(data2)
set.seed(1)
folds <- sample(rep(1:k, length = n))
cv.errors <- matrix(NA, k, 11, dimnames = list(NULL, paste(1:11)))
```

there is no predict method for the library so we implemented one

```
predict.regsbsets = function(object, newdata, id, ...) {
  form = as.formula(object$call[[2]])
  mat = model.matrix(form, newdata)
  coefi = coef(object, id = id)
  mat[, names(coefi)] %*% coefi
}
#test= j #train=i
for (j in 1:k) {
  best.fit <- regsubsets(quality~., data = data2[folds != j, ], nvmax = 11)
  for (i in 1:11) {
    pred <- predict(best.fit, data2[folds == j, ], id = i)
    cv.errors[j, i] <- mean((data2$quality[folds == j] - pred)^2)
  }
}
```

Find the MSE

```
mse <- apply(cv.errors, 2, mean)
#plot
par(mfrow = c(1, 1))
plot(mse, type = "b")
```



```
min <- which.min(msep)
cross_mse <- msep[min]
```

The best subset selection based on the MSE is a 10-variable model that excludes the variables citric.acid and total.sulfur.dioxide.

Shrinkage models: RIDGE AND LASSO

First we compute the model matrices to use in both the models and divide in training and test set

```
dat = as.data.frame(data) #>%>%dplyr::select(-quality)
X <- model.matrix(quality~.-1, data = dat)
Y <- dat$quality
Xtr <- model.matrix(quality~.-1, data = d_ft)
Ytr <- d_ft$quality
Xte <- model.matrix(quality~.-1, data = d_ftest1)
Yte <- test_quality
```

LASSO

```
#we fit the model
lasso.fit <- glmnet(Xtr, Ytr, alpha=1)
```

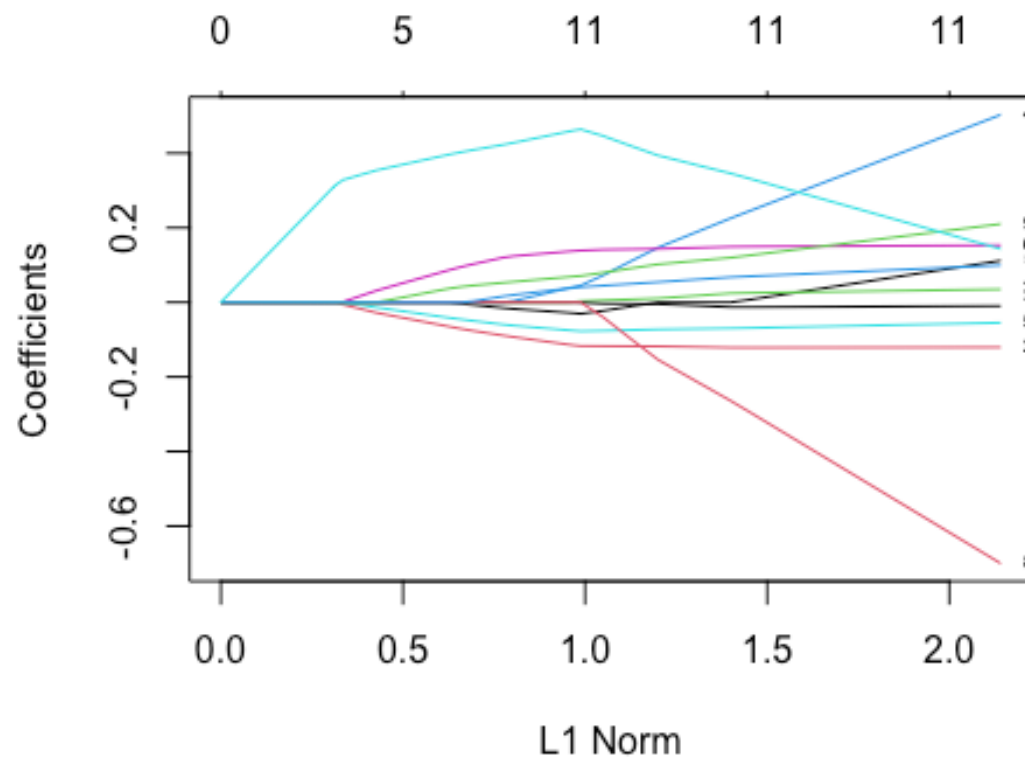
the set of coefficients estimates $\hat{\beta}$ for the sequence of lambda provided by glmnet

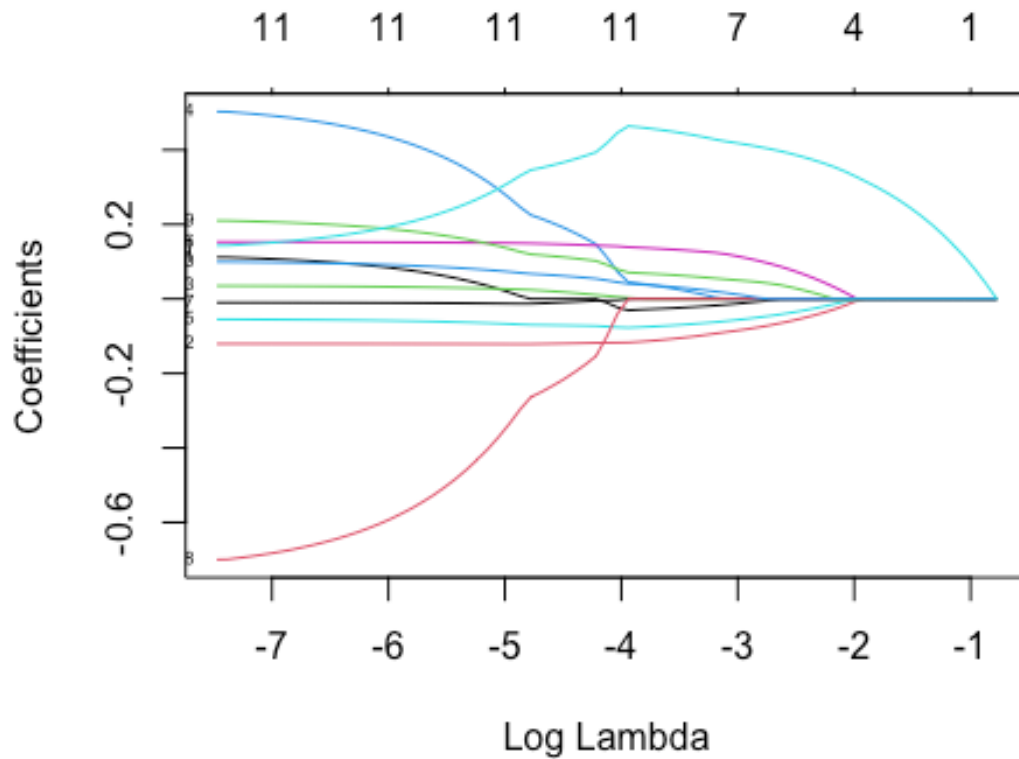
```
lasso.fit$beta
```

```
##    [[ suppressing 73 column names 's0', 's1', 's2' ... ]]
```

```
lasso.fit$lambda
```

We plot the lasso regression coefficients as a function of $\log(\lambda)$ and of the L1 norm

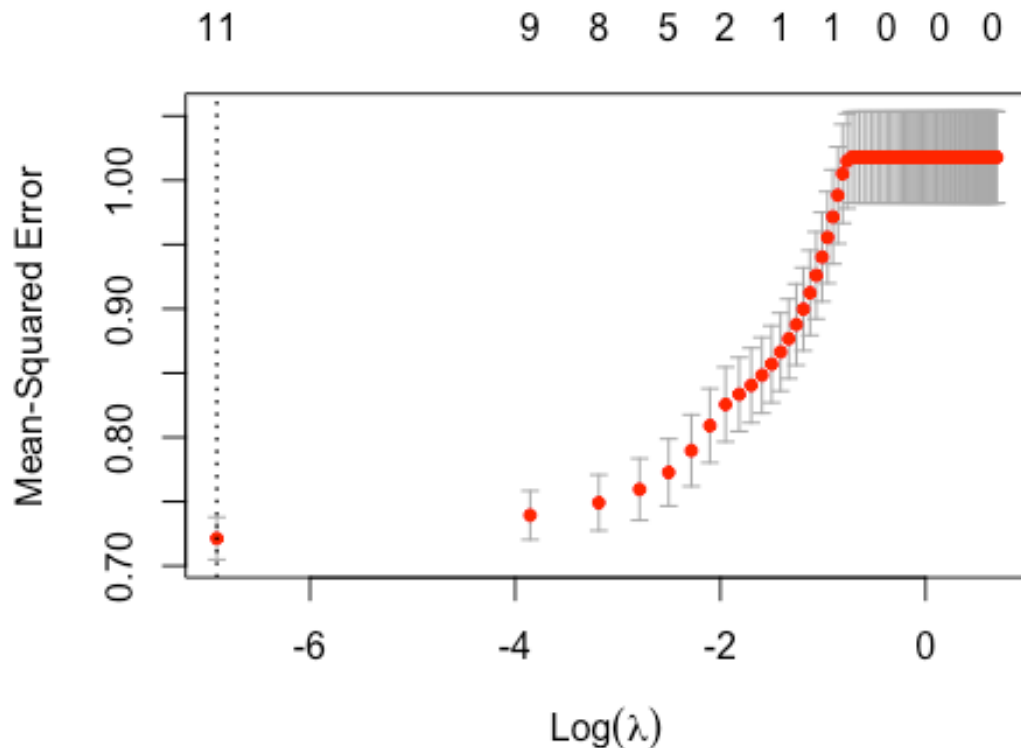




We can see from these plots that depending on the value of the tuning parameter λ , some of the coefficients will be shrunk to 0.

We now want to pick the best λ , we're going to do it using k-fold using $k=5$

```
Lasso.cv <- cv.glmnet(Xtr, Ytr, lambda=seq(0.001, 2, length.out = 100), alpha
=1, nfolds=5,
                    family="gaussian")
plot(Lasso.cv)
```



Through Cross validation we find the best lambda and then perform the predictions

```
bestlam <- Lasso.cv$lambda.min
```

Prediction on the test set and MSE

```
Lasso.pred <- predict(lasso.fit, s=bestlam, newx=Xte)
mse_Lasso<-mean((Lasso.pred-Yte)^2)
```

And now we show the coefficients for the chosen lambda

```
out <- glmnet(X,Y, alpha=1)
lasso.coef <- predict(out, type="coefficients", s= bestlam)[1:12,]
round(lasso.coef,2)
```

```
##      (Intercept)      fixed.acidity      volatile.acidity
##           0.00           0.10           -0.13
##      citric.acid      residual.sugar      chlorides
##           0.03           0.43           -0.07
## free.sulfur.dioxide total.sulfur.dioxide      density
##           0.14           -0.02           -0.57
##           pH           sulphates      alcohol
##           0.18           0.09           0.21
```

```
lasso.coef <-lasso.coef[lasso.coef!=0]
```


RIDGE

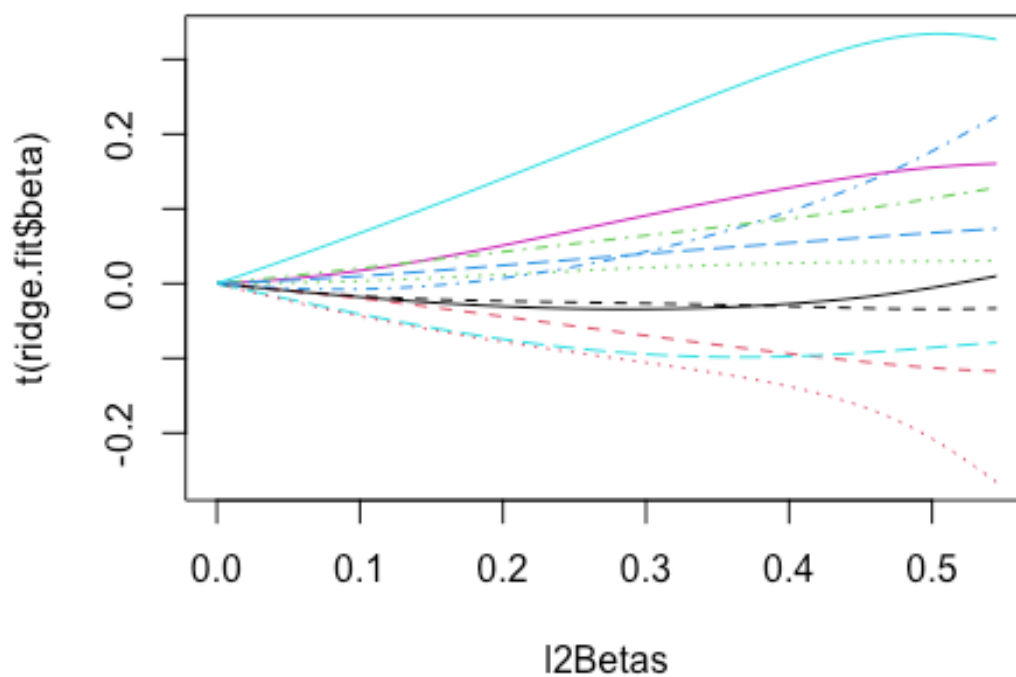
As in lasso we do cross validation to find best lambda. To perform the ridge the procedure is the same but we need to set alpha to 0

```
ridge.fit <- glmnet(Xtr, Ytr, alpha=0)
ridge.fit$beta

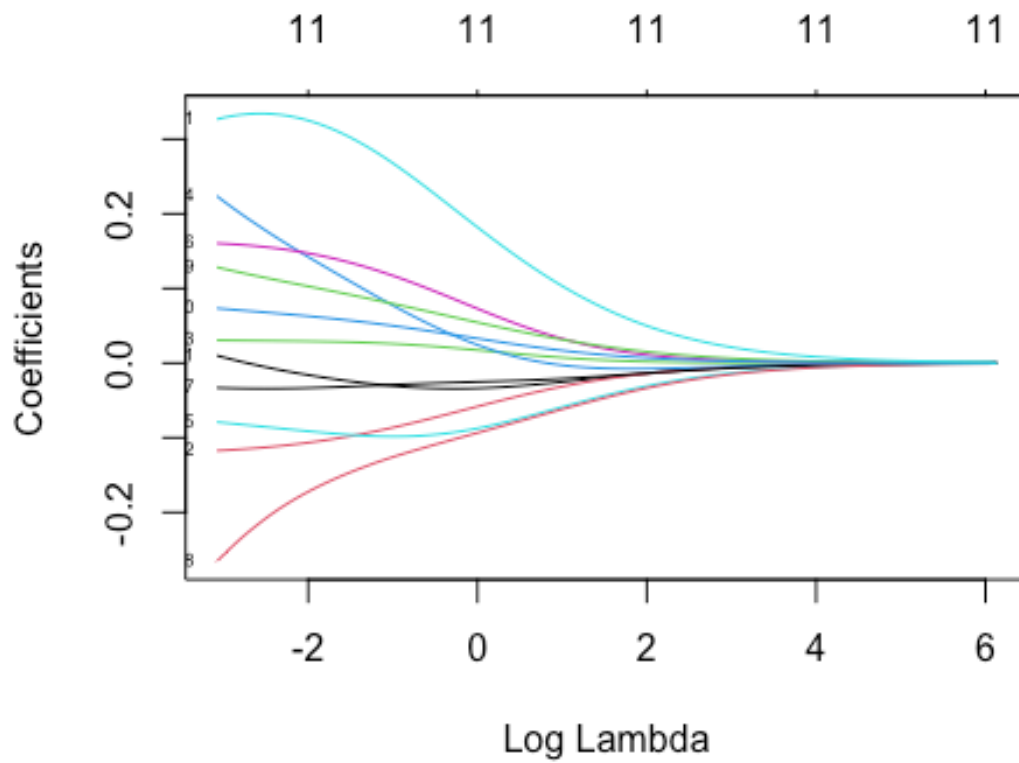
##    [[ suppressing 100 column names 's0', 's1', 's2' ... ]]

ridge.fit$lambda
```

We now plot the ridge regression coefficients as a function of the L2 norm

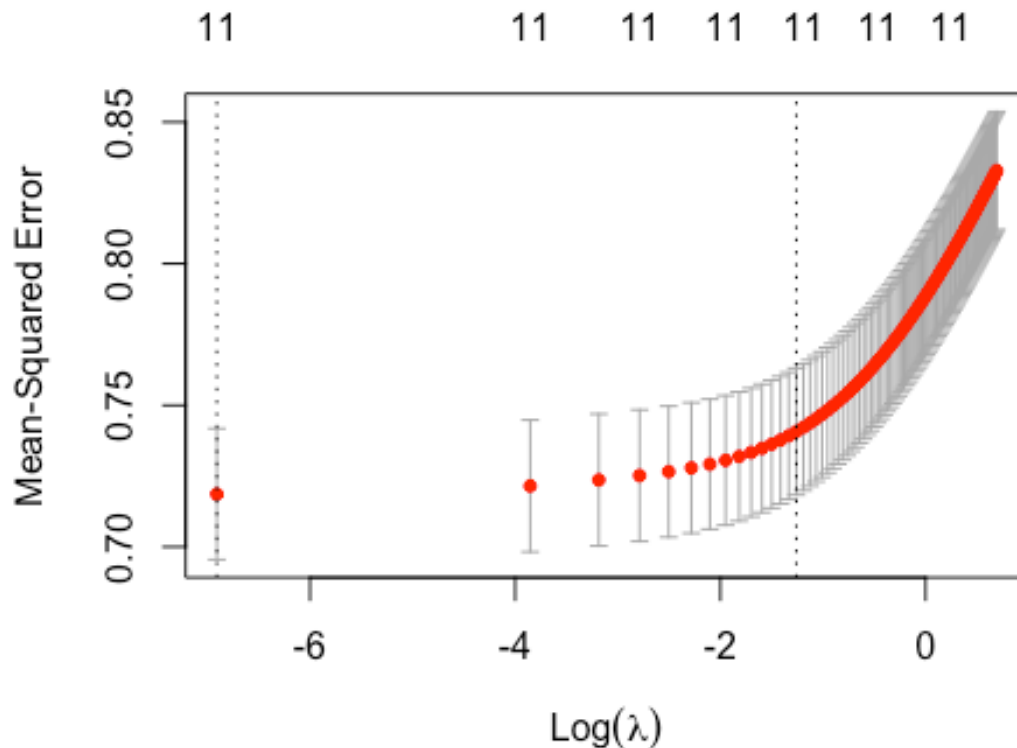


and of $\log(\lambda)$



k-fold validation to find the best lambda

```
Ridge.cv <- cv.glmnet(Xtr, Ytr, lambda=seq(0.001, 2, length.out = 100), alpha
=0, nfolds=5,
                    family="gaussian")
plot(Ridge.cv)
```



We then pick the lambda that minimizes the MSE

```
bestlam <- Ridge.cv$Ridge.min
```

We can conclude that the shrinkage models that yield the lowest MSE is the Lasso

Afterward we begin with the nonlinear models:

Splines

We use manual cross validation to find the degrees of freedom that minimize the MSE and compare Natural Splines, Bsplines and Polynomials

```
nSpars <- 100
spars <- seq(0.1, 0.99, length.out=nSpars)
```

approximation of cross validation scores

```
gcvScores <- matrix(NA, nrow=2, ncol=length(spars))
```

Use 3 degrees

```
deg_free <- 3
```

We then plot the MSEs for every knot

```

bKnots <- range(data2$alcohol)
# Folds
nfolds <- 10
n <- nrow(data2)
foldid <- sample(1:nfolds, n, replace = T)
# Looping
cvScores <- matrix(NA, nrow=3, ncol=nfolds)
rownames(cvScores) <- c("Polynomial", "Natural Splines", "B-Spline")
for (i in 1:nfolds)
{
  datIn <- data2 %>% filter(foldid!=i)
  datOut <- data2 %>% filter(foldid==i)

  oo1 <- lm(quality~poly(alcohol, degree = deg_free), data=datIn)
  pred1 <- predict(oo1, datOut)
  cvScores[1, i] <- mean((pred1-datOut$quality)^2)
  oo2 <- lm(quality~ns(x=alcohol, df = deg_free, Boundary.knots = bKnots), da
ta=datIn)
  pred2 <- predict(oo2, datOut)
  cvScores[2, i] <- mean((pred2-datOut$quality)^2)
  oo3 <- lm(quality~bs(x=alcohol, df = deg_free, Boundary.knots = bKnots), da
ta=datIn)
  pred3 <- predict(oo3, datOut)
  cvScores[3, i] <- mean((pred3-datOut$quality)^2)
}

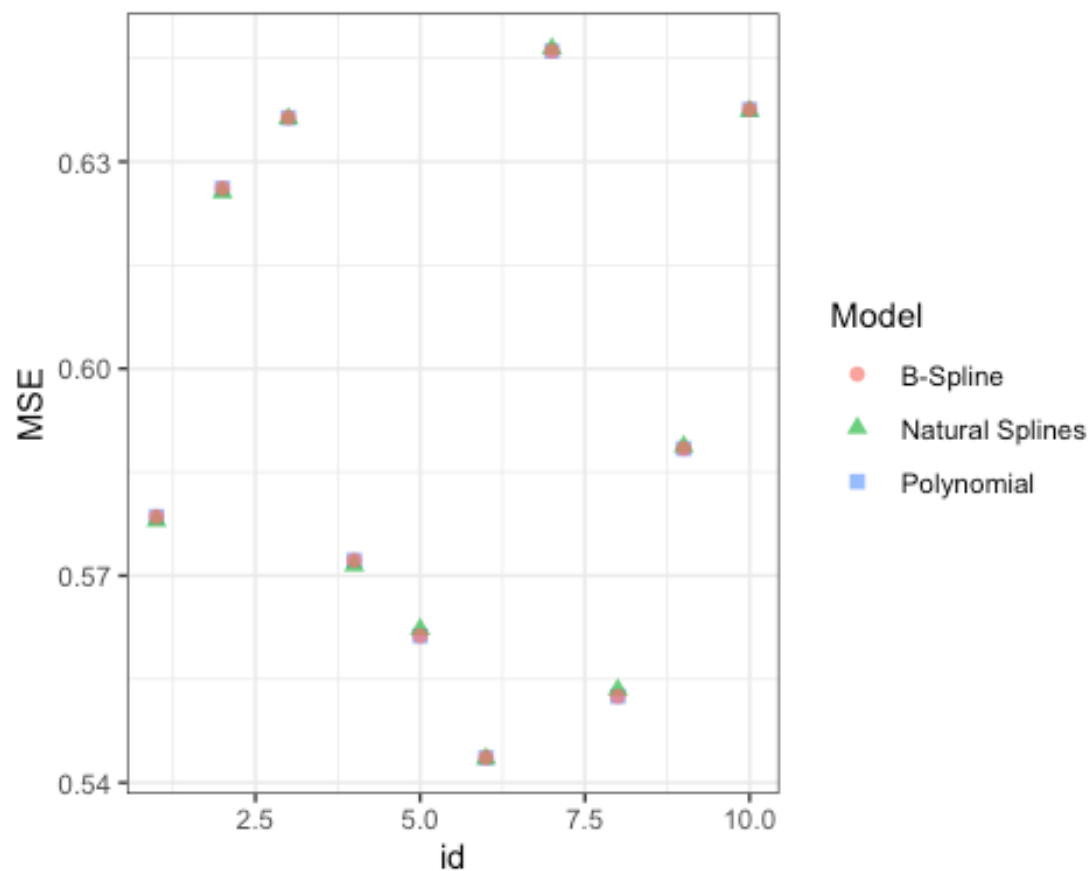
```

Errors on each fold for every model

```

t(cvScores) %>% as_tibble() %>% mutate(id=1:nrow(.)) %>% gather(Model, MSE, -
id) %>%
  ggplot() + geom_point(aes(x=id, y=MSE, col=Model, shape=Model), size=2, alp
ha=0.7) + theme_bw()

```

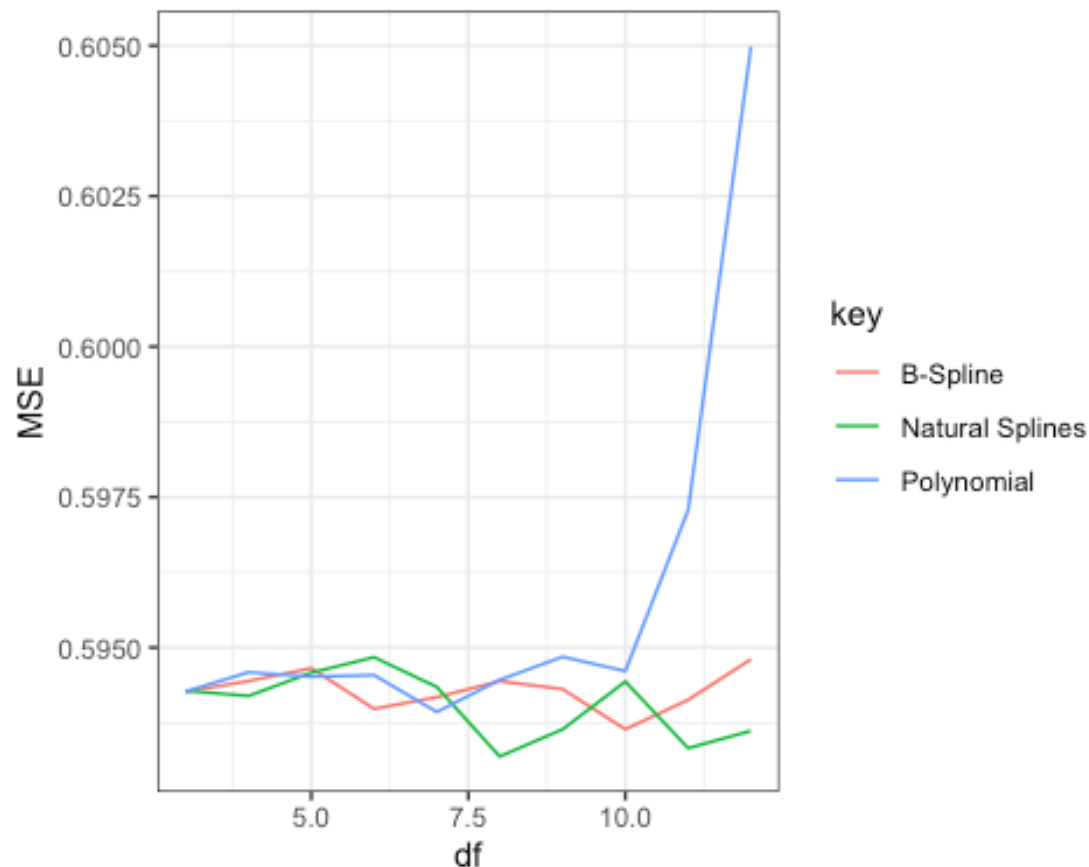


Average error for every method used, we see that Natural Splines have Least error

```
rowMeans(cvScores)
```

```
##      Polynomial Natural Splines      B-Spline
##      0.5942684      0.5942819      0.5942684
```

Now we evaluate the mean square error of each model for different degrees of freedom



The best choice of degrees of freedom seems to be somewhere between 5 and 10 we can go a bit further with degree of freedom for natural splines with too much complexity polynomial is the one performing worse cause it has non additional constraints

We pick for natural is at 11 dfs and for b-splines is 10. For polynomials around 7

So then we fit the models and perform the predictions

```
## Warning in bs(alcohol, degree = 3L, knots = c(`12.5%` = -1.18216345974233,
:
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

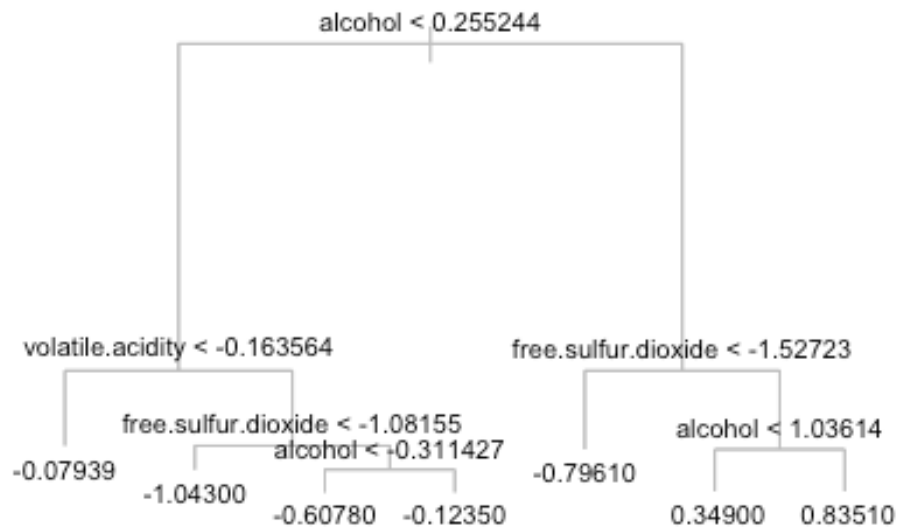
Last model for predictions is the

Decision trees

```
## Registered S3 method overwritten by 'tree':
## method      from
## print.tree cli

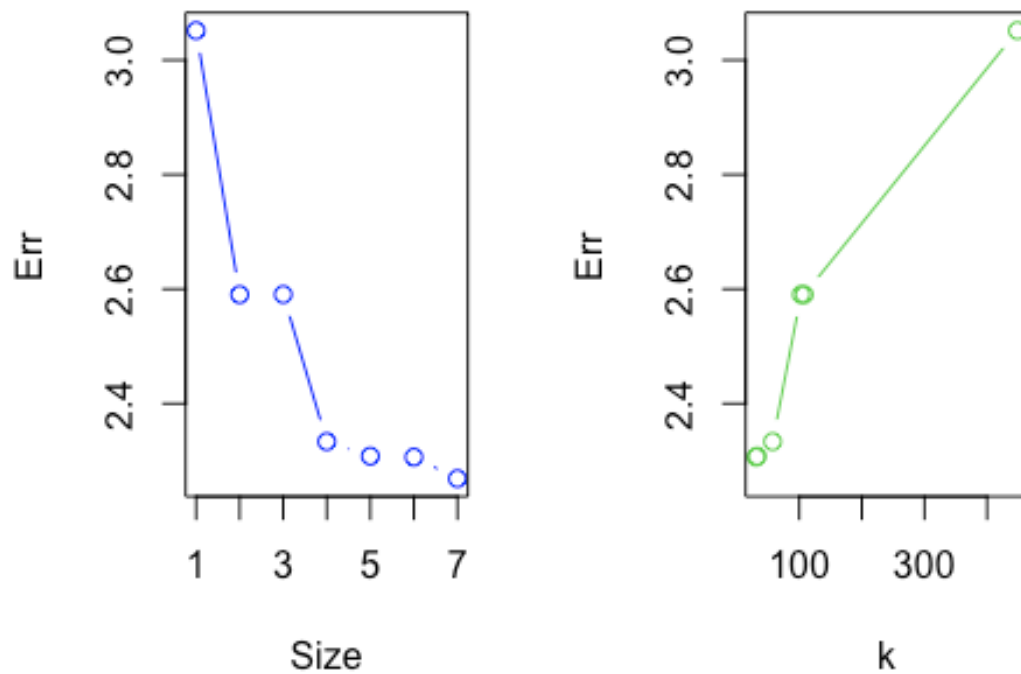
#Regression tree fit
fit_tr = tree(quality~., data=d_ft)
summary(fit_tr)
plot(fit_tr, col="grey")
```

```
text(fit_tr, pretty=0, cex = 0.7)
```



```
#make the predictions  
pred_test = predict(fit_tr, d_ftest)
```

We want to see if tree pruning improves the performance of the tree: to do so we run a K-fold cross-validation and we evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of k.



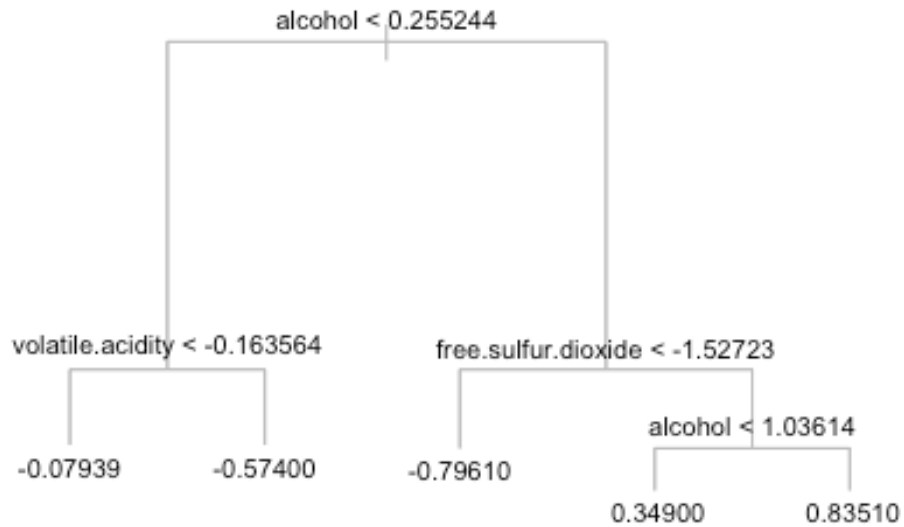
From the plot we can see that the pruning would not benefit to the performance of the model, but we perform the tree pruning anyways (to confirm our assumption) with tree size=5 and pruning parameter=50

Pruning

```
fit_pr = prune.tree(fit_tr, best = 5, k=50)
par(mfrow=c(1,1))
plot(fit_pr, col="grey")
```



```
text(fit_pr, pretty=0, cex = 0.7)
```



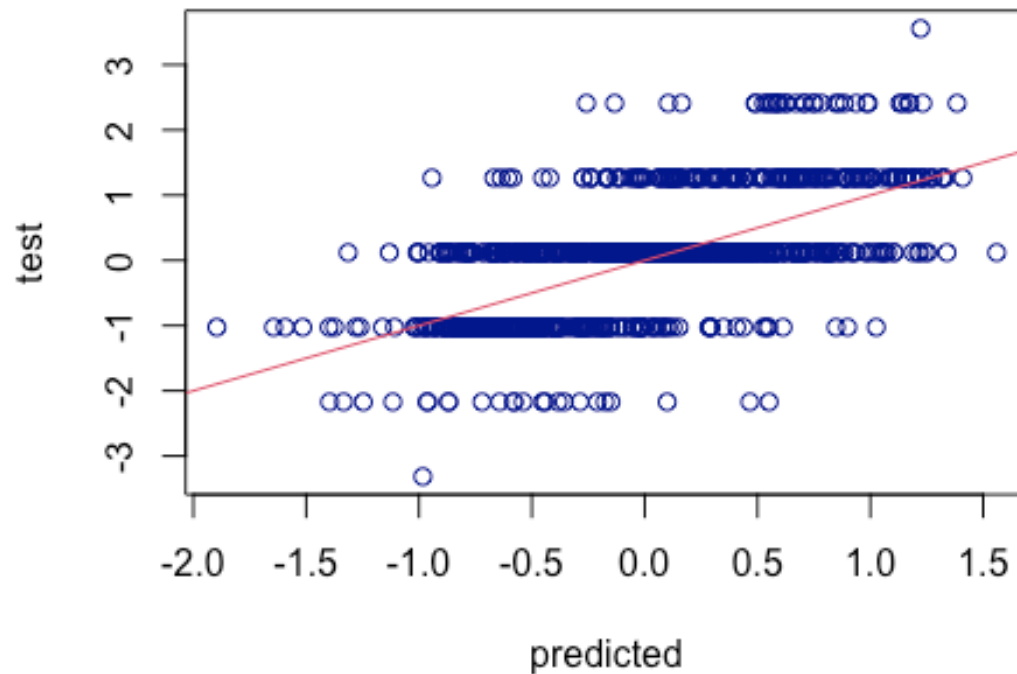
We use our pruned tree to make predictions on the test set and compute the MSE.

Then we do the Random Forests: starting from all the predictors (bagging)

Bagging

We perform bagging by exploiting the randomForest function and considering all of our predictors for each split of the tree

```
fit_bag = randomForest(quality~., data = d_ft, mtry = ncol(d_ft)-1, importance = T)
```



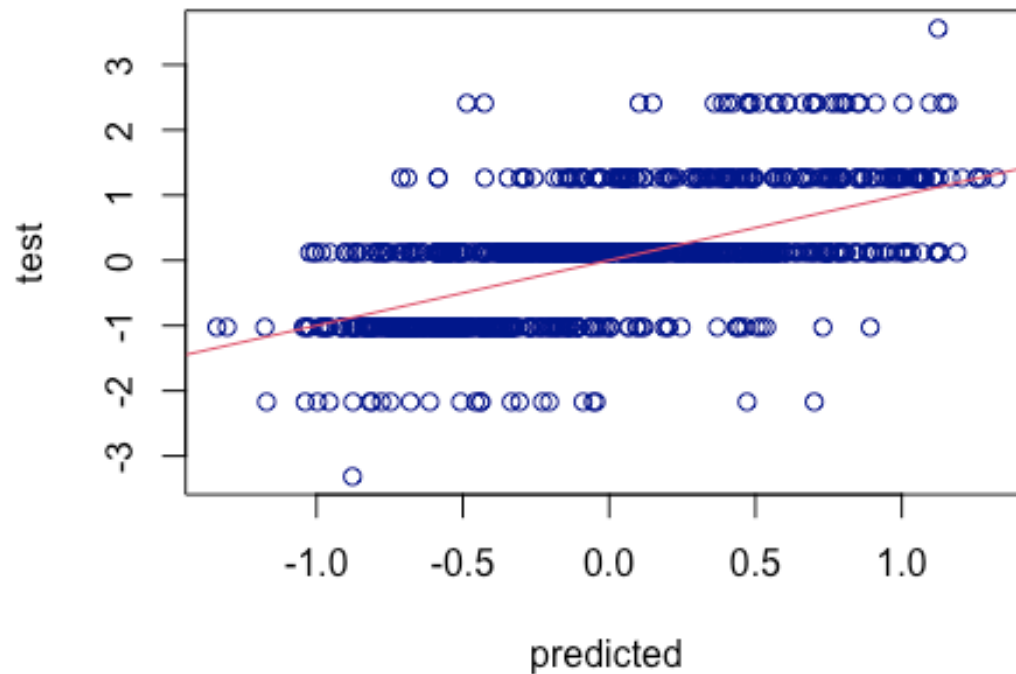
The test set MSE associated with the bagged regression tree is lower than the one obtained with the regression tree.

Random forest

Then we fit a random forest with just 2 predictors

```
fit_rf = randomForest(quality~., data = d_ft, mtry = 2, importance = T, ntree = 1e3 )
```

After we make the predictions and evaluate the MSE.



the MSE of the random forest yields an improvement over the bagging.

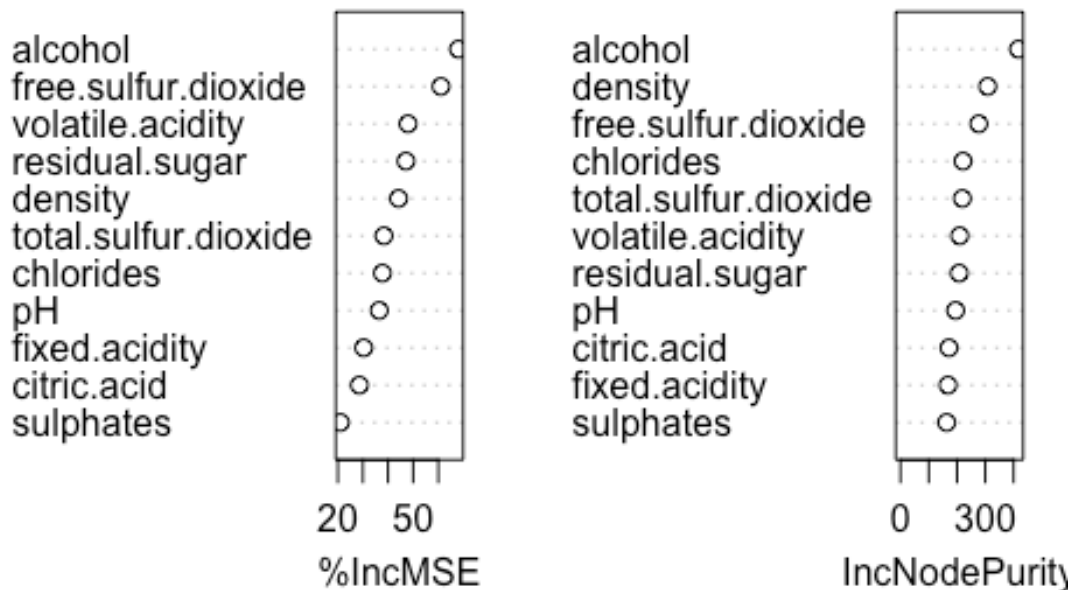
Plot the importance of each variable

```
importance(fit_rf)
```

##	%IncMSE	IncNodePurity
## fixed.acidity	30.31996	168.1220
## volatile.acidity	47.97173	209.5644
## citric.acid	28.60757	171.6048
## residual.sugar	47.15318	207.6898
## chlorides	37.78251	221.1261
## free.sulfur.dioxide	61.08437	277.0079
## total.sulfur.dioxide	38.37024	218.8337
## density	44.20055	308.4086
## pH	36.51379	195.1077
## sulphates	21.03982	163.5324
## alcohol	68.05630	417.0587

```
varImpPlot(fit_rf)
```

fit_rf



The alcohol is the most important variable followed by the free.sulfur.dioxide.

Point 6

Compare the results

```
vector<-c(msebic,mse_aic,mse_bic,cross_mse,mse_Ridge,mse_Lasso,msebsplines,ms
ensplines,msepolynomials,msedecisiontree,msepruning, msebagging, mse_randomf)
vector
```

```
##                                10
## 0.6865987 0.8989521 0.8173199 0.5388563 0.8142569 0.6757247 0.7149075 0.71
22630
##
## 0.7157628 0.7028344 0.7116510 0.5962608 0.5938442
```

```
min(vector)
```

```
## [1] 0.5388563
```

the model with the least MSE is the model fitted with the cross validation selection knit

Conclusion:

By evaluating the MSE on the test set for every model, we observed that the best model is the cross-validation model. It selects a linear multiple regression with 10 predictors (every variable except for total.sulfur.dioxide) with a MSE of approximately 0,54. So we can conclude that our data can be interpreted through a linear model.

Task 2

Introduction:

At this point of our analysis we already have developed a regression predictive algorithm to assess the quality of an "in working wine".

In a business perspective, to give a firm a full understanding of which chemical characteristics make a "good wine", we performed a clustering. We divided our response vector "quality" in a binary factor ("bad wine" and "good wine") and compared it with the clustering results. Finally we evaluated the most meaningful predictors for the clustering.

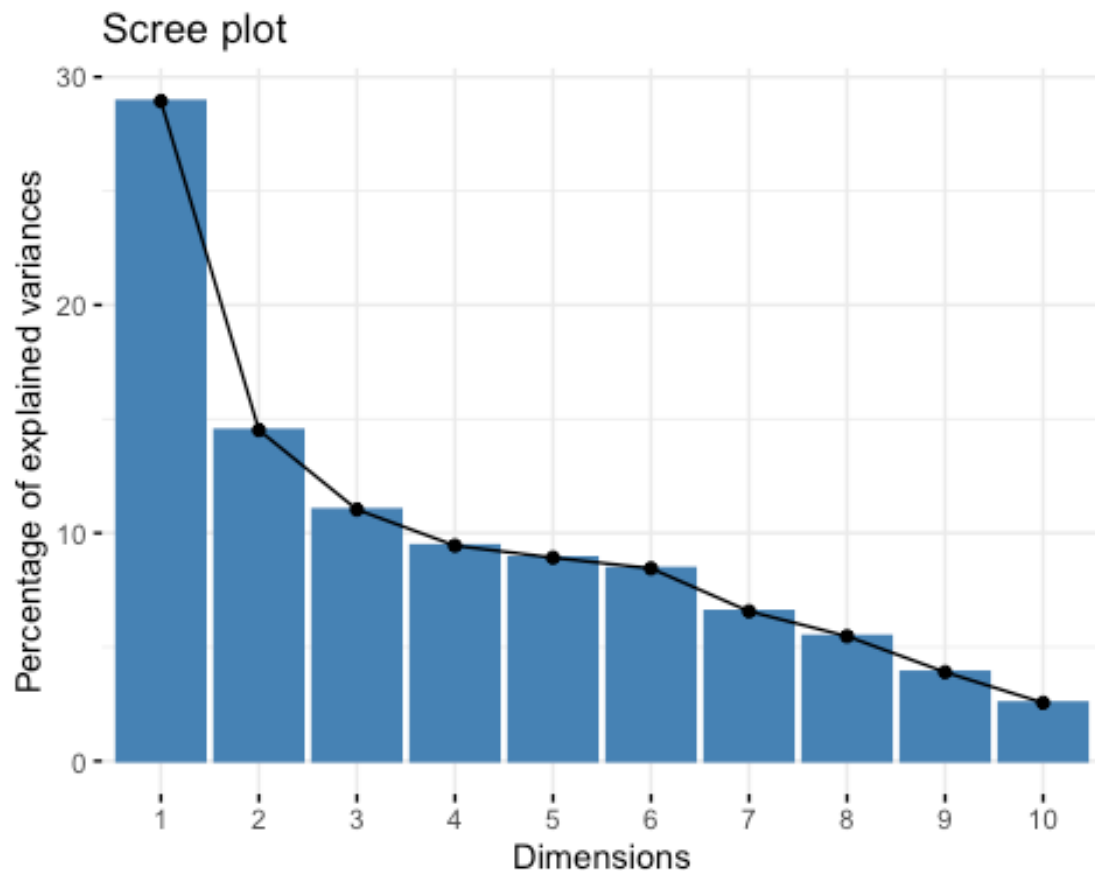
Clustering

We first clean the dataset from any duplicate or NA value and then we bin the quality in bad wines (quality score<6) and good wines (quality score>=6).

Then we perform PCA on the clean dataset (without the response) in order to shape the best parameters to perform the clustering.

PCA

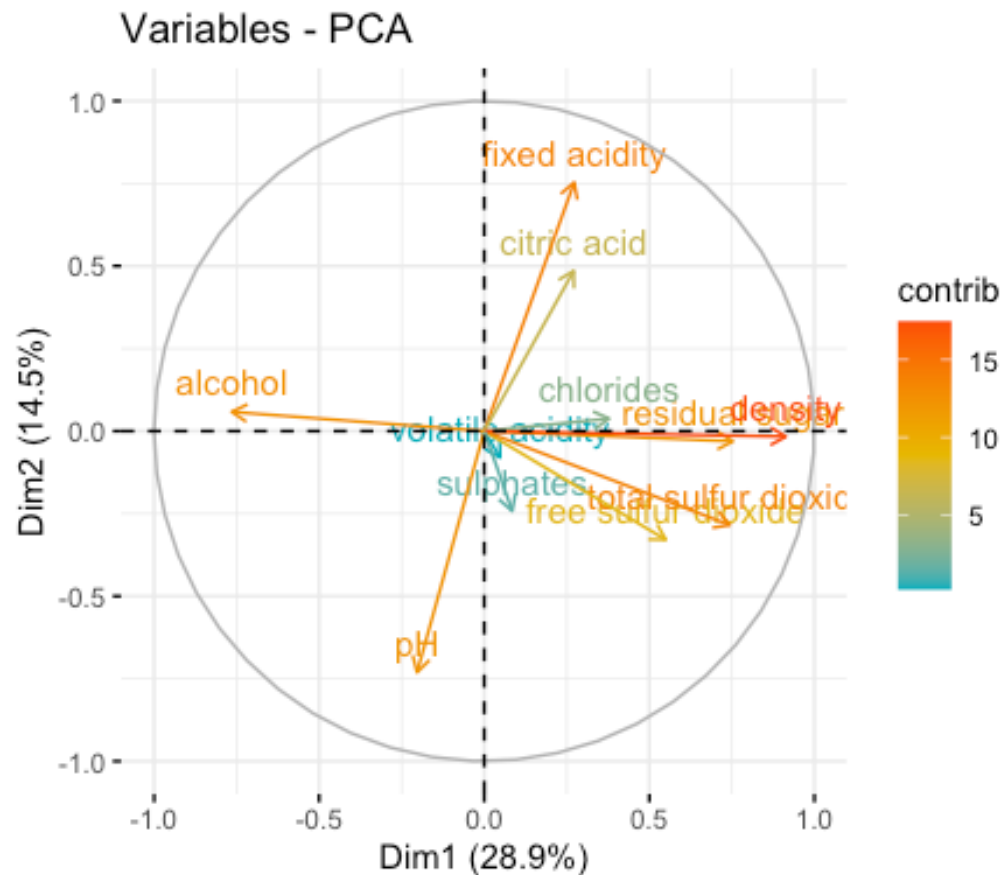
```
#scale the variables
apply(data_noqual, 2, mean)
apply(data_noqual, 2, var)
#There is no need to scale the data since we put "cor=T"
pca = princomp(data_noqual, cor=T) #Apply the principal components analysis using correlation matrix
summary(pca)
fviz_eig(pca)
```



```
pc.comp <- pca$scores
pc.comp
pc.comp1 <- -1*pc.comp[,1] #We extract principal component 1 scores
pc.comp2 <- -1*pc.comp[,2]
pc.comp3 <- -1*pc.comp[,3]
pc.comp4 <- -1*pc.comp[,4]
pc.comp5 <- -1*pc.comp[,5]
data_pca <- data_noqual
data_pca$pc.comp1 <- pc.comp1
data_pca$pc.comp2 <- pc.comp2
data_pca$pc.comp3 <- pc.comp3
data_pca$pc.comp4 <- pc.comp4
data_pca$pc.comp5 <- pc.comp5
data_pca <- data_pca %>% dplyr::select(pc.comp1, pc.comp2, pc.comp3, pc.comp4, pc.comp5)
data_pca
```

Then we look at which variables contribute more to the components

```
fviz_pca_var(pca,
  col.var = "contrib",
  gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
)
```



Eigenvalues

```
eig.val <- get_eigenvalue(pca)
eig.val
```

Results for Variables

```
res.var <- get_pca_var(pca)
res.var$coord # Coordinates
```

	Dim.1	Dim.2	Dim.3	Dim.4
## fixed acidity	0.27269588	0.75303655	0.06108969	0.003124276
## volatile acidity	0.04776586	-0.08003547	-0.74034852	0.171504141
## citric acid	0.27104607	0.48522968	0.50823689	0.145063472
## residual sugar	0.75312133	-0.03097378	-0.22272688	-0.285498538
## chlorides	0.37820238	0.03752709	-0.11959338	0.682356858
## free sulfur dioxide	0.54982038	-0.32905066	0.34134241	-0.344151068
## total sulfur dioxide	0.74368505	-0.28454944	0.13590691	-0.095729362
## density	0.91360233	-0.01655750	-0.14190667	0.006318739
## pH	-0.20343973	-0.73065204	0.15970583	0.136720814
## sulphates	0.08464282	-0.24173161	0.38858427	0.507559422
## alcohol	-0.76459508	0.05811486	0.08577522	-0.193667589
	Dim.5	Dim.6	Dim.7	Dim.8
## fixed acidity	0.257588259	0.002740261	0.17936932	0.43755274
## volatile acidity	0.380214294	0.417458359	-0.24232229	0.08384854


```
## citric acid      0.000848936  0.116297739 -0.60427587 -0.05599964
## residual sugar   0.157508972 -0.253766921 -0.20495778 -0.32755727
## chlorides        -0.482443784  0.235787583  0.01954078 -0.10629806
## free sulfur dioxide -0.098836299  0.477629971  0.12135420 -0.04510239
## total sulfur dioxide 0.108815620  0.362647809  0.04719207  0.21267732
## density          0.084568499 -0.337316942 -0.06801306  0.03902702
## pH               -0.003482202 -0.248451609 -0.34063619  0.40843353
## sulphates        0.652387558 -0.060985068  0.22307583 -0.21263957
## alcohol          0.241394769  0.287450958 -0.19036600 -0.14493321
##                Dim.9      Dim.10      Dim.11
## fixed acidity     0.242629129  0.07220149  0.027645632
## volatile acidity  -0.117862704  0.12368248  0.003017868
## citric acid       -0.170917643  0.02666815  0.001684013
## residual sugar     0.234835109 -0.03834425  0.074874685
## chlorides         0.273513006 -0.02804955  0.004021042
## free sulfur dioxide 0.082049711  0.30504530 -0.004873492
## total sulfur dioxide -0.072355155 -0.37716779  0.005054842
## density           0.052257261  0.03972286 -0.116133713
## pH                0.176629596  0.06530967  0.022509476
## sulphates         -0.006419667  0.03734975  0.006617028
## alcohol           0.388544022 -0.12198528 -0.056225404

res.var$contrib # Contributions to the PCs
res.var$cos2    # Quality of representation

# Results for individuals
res.ind <- get_pca_ind(pca)
res.ind$coord # Coordinates
res.ind$contrib # Contributions to the PCs
res.ind$cos2 # Quality of representation
```

k-Means Clustering

Silhouette and elbow-method on k-Means find that the optimal number of clusters is 2.

```
km2 <- kmeans(data_pca, centers = 2)
km2

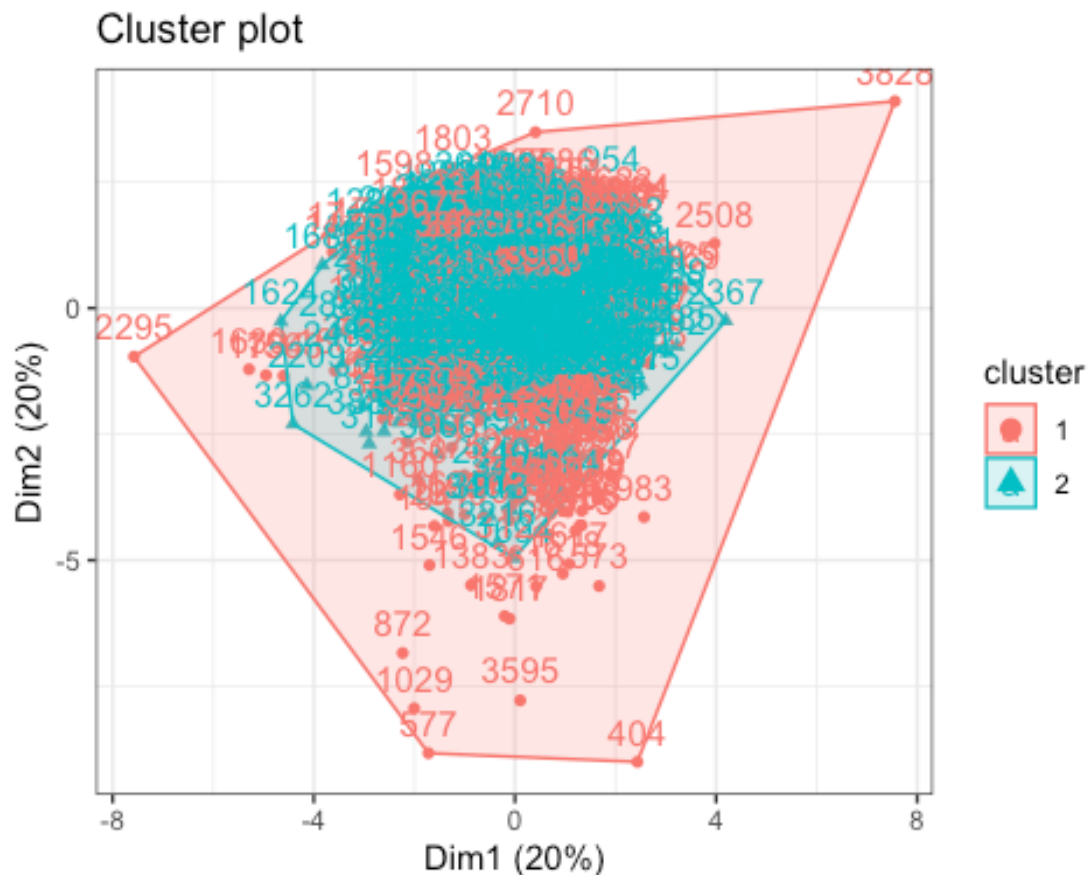
## K-means clustering with 2 clusters of sizes 1486, 2475
##
## Cluster means:
##      pc.comp1    pc.comp2    pc.comp3    pc.comp4    pc.comp5
## 1 -1.921356    0.11281346  0.03478809  0.04874152  0.03162125
## 2  1.153590   -0.06773366 -0.02088691 -0.02926460 -0.01898552
##
##
## Within cluster sum of squares by cluster:
## [1] 9678.732 13241.101
## (between_SS / total_SS = 27.8 %)
##
## Available components:
```

```
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withi
nss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

The main difference between the two clusters is that in cluster 1 (bad wine) the influence of the first component is negative while in cluster 2 (good wine) the influence of the first component is the only one that is positive.

Clusters plot

```
fviz_cluster(object=km2,data_pca) + theme_bw()
```



Now we compare if the clustering on the considered numerical variables result of the pca agrees with one of the original categorical variable: Quality (good wine or bad wine)

```
datWC <- data_pca %>% mutate(clust=km2$cluster)
```

We can show a contingency table

```
datWC$quality <- qual
(contTable <- datWC %>% dplyr::select(quality, clust) %>% table())
```

```
##          clust
## quality    1    2
```

```
##    bad wine    725   623
##    good wine   761 1852

chisq.test(contTable) #p-value is low

##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data:  contTable
## X-squared = 229.63, df = 1, p-value < 2.2e-16

prop.table(contTable, margin=1)

##           clust
## quality      1      2
##    bad wine  0.5378338 0.4621662
##    good wine 0.2912361 0.7087639
```

“good wine” and “bad wine” have not so different distribution across the first clusters, while in the second cluster around 3/4 of data points have good wine prediction

```
mclust::adjustedRandIndex(datWC$quality, datWC$clust)

## [1] 0.08469879
```

Since ARI is low (0.085) in the cluster labels the wines tend to belong to the same cluster even if they have not similar characteristics

Conclusion:

Unfortunately the result of the ARI index shows that the 2 clusters don't tend to show evident propensity for having good quality or bad quality in wines. However, analyzing the pca components and the variables from which they are composed of, we can state that the first component is the one with more weight. This component is for the most influenced by residual.sugar, density, total.sulfur.dioxide and alcohol. So we can deduce that negative values for the first component tend to worsen the quality of the wine, conversely positive values lead to good wines.

This clustering result isn't shocking as in Task 1 we underlined how this problem is interpretable through a linear regression of 10 variables. Anyway, the implementation of the Clustering turned out to be very useful as it confirms the importance of the predictors we selected in Task 1.